JS

Курс из 3 частей · для тех, кто уже знает C# и Python

# Данные оживают

## Часть 2. JSON, поиск, фильтр, сортировка, кнопки

**ЧАСТЬ 1 · ПРОЙДЕНО**

Массив в script.js → render(list)

**ЧАСТЬ 2 · СЕЙЧАС**

data.json + кнопки, поиск и сортировка

**ЧАСТЬ 3**

Удаление, localStorage, финальный проект

ТЕОРИЯ · 1

# JSON — объект, записанный текстом

КОД · script.js

```
// data.json — это просто ТЕКСТ такого вида:
// [{ "id": 1, "title": "Наушники", "price": 2990 }]

const product = { id: 1, title: "Наушники" };
const text = JSON.stringify(product); // → текст
console.log(text);
console.log(typeof text);

const back = JSON.parse(text); // → объект
console.log(back.title);
```

## ЧТО ПРОИЗОЙДЁТ

▸ Console

```
{"id":1,"title":"Наушники"}
string
Наушники
```

**JSON строже, чем объект JS:**

- ключи — **только в "двойных" кавычках**
- строки — тоже только в " ", не в ' '
- **нет запятой** после последнего элемента
- нет комментариев и функций

**Как это в C# / Python:** json.dumps / json.loads в Python, JsonSerializer.Serialize / Deserialize в C#.

ТЕОРИЯ · 2

# Загружаем data.json: fetch + async/await

КОД · script.js

```
// async — внутри функции можно ждать (await)
async function loadProducts() {
  // 1) попросили файл и ждём ответ
  const response = await fetch("data.json");
  // 2) текст ответа → массив (это тоже ждём)
  const data = await response.json();
  console.log(data.length, data[0].title);
  render(data); // 3) рисуем, как раньше
}

loadProducts();
```

## ЧТО ПРОИЗОЙДЁТ

```
▸ Console
6 'Наушники'
```

...и на странице 6 карточек из файла. **await** = «подожди ответа и только потом иди дальше». Без async снаружи await писать нельзя.

**⚠ Ловушка JS: забытый await**

```
const data = response.json();
data // Promise {<pending>}
data.length // undefined
```

Promise — «обещание данных», а не сами данные.

**Как это в C# / Python:** один в один как async Task + await client.GetStringAsync(...) в C# или async def + await в Python.

⚠ ВАЖНО ДЛЯ КОНКУРСА

# Почему fetch не работает по двойному клику

**Не работает: двойной клик**

```
file:///C:/site/index.html
```

```
▸ Console
Access to fetch at 'file:///…/data.json' from
origin 'null' has been blocked by CORS policy
```

Браузер защищает тебя: страница, открытая прямо с диска, **не может читать другие файлы** через fetch. Иначе любой скачанный HTML читал бы твои документы.

**Работает: Live Server в VS Code**

1. Extensions (Ctrl+Shift+X) → найди **Live Server** → Install.
2. Правой кнопкой по index.html → **Open with Live Server**.
3. Адрес теперь **http://127.0.0.1:5500** — это мини-сервер, fetch работает.
4. Бонус: сохранил файл → страница обновилась сама.

Запасной план: данные прямо в script.js

```
const data = [ /* вставь data.json */ ];
render(data);
```

**Как это в Python:** то же, что запустить в папке python -m http.server и открыть localhost:8000.

ТЕОРИЯ · 3

# filter и map: отобрать и превратить

КОД · products — 6 товаров из data.json

```
// filter — оставить подходящие (как Where)
const cheap = products.filter((p) ⇒ p.price < 3000);
console.log(cheap.length);

// map — превратить каждый (как Select)
const titles = products.map((p) ⇒ p.title);
console.log(titles);

// цепочкой: сначала отобрать, потом превратить
const cheapTitles = products
  .filter((p) ⇒ p.price < 3000)
  .map((p) ⇒ p.title);
```

## ЧТО ПРОИЗОЙДЁТ

```
▸ Console
3
▸ ['Наушники', 'Мышь', 'Клавиатура',
   'Колонка', 'Геймпад', 'Руль']
```

**БЫЛО → filter(цена < 3000) → СТАЛО**

**Наушники 2990 ✓** **Мышь 1490 ✓** Клавиатура 3990

**Колонка 2490 ✓** Геймпад 3490 Руль 5990

**(p) => p.price < 3000** — «для каждого товара p скажи true или false». filter оставит только true. Исходный products **не меняется**.

**Как это в C# / Python:** LINQ Where / Select, или [p for p in products if ...] и [p["title"] for p in products].

ТЕОРИЯ · 4

# find, includes, sort и распаковка { }

КОД · тот же products

```
// find — первый подходящий (FirstOrDefault)
const item = products.find((p) ⇒ p.id === 2);
const { title, price } = item; // распаковка
console.log(title, price);

// includes — есть ли кусок текста в строке
console.log("Клавиатура".includes("лав"));

// sort — по возрастанию цены, на КОПИИ
const sorted = [...products]
  .sort((a, b) ⇒ a.price - b.price);
console.log(sorted[0].title);
```

## ЧТО ПРОИЗОЙДЁТ

▸ Console

```
Мышь 1490
true
Мышь
```

**const { title, price } = item** — достаём поля в одноимённые переменные. То же, что title = item.title и price = item.price.

**(a, b) => a.price - b.price** — минус: a раньше, плюс: b раньше. Дорогие сначала: **b.price - a.price**.

⚠ **Осторожно:** sort меняет сам массив. [...products] — быстрая копия, чтобы не испортить оригинал.

**Как это в C# / Python:** FirstOrDefault, OrderBy(p => p.Price), Contains · next(...), sorted(key=...), "лав" in s.

ТЕОРИЯ · 5

# События: когда случится X — сделай Y

КОД · script.js

```
// HTML: <button id="hello">Нажми меня</button>
const btn = document.querySelector("#hello");
btn.addEventListener("click", () ⇒ {
  console.log("Клик!");
});

// HTML: <input id="search">
const search = document.querySelector("#search");
search.addEventListener("input", () ⇒ {
  console.log("Ввели:", search.value);
});
```

## ЧТО ПРОИЗОЙДЁТ




Нажали кнопку, потом напечатали «мы»:

```
▸ Console
Клик!
Ввели: м
Ввели: мы
```

**click** — нажатие, **input** — каждая буква, **change** — выбор в <select>. **search.value** — текст в поле.

**Как это в C# / Python:** btn.Click += (s, e) => { ... } в C# — та же идея! В tkinter: Button(command=on_click).

ПРАКТИКА · ШАГ 1 ИЗ 3

# Мозг каталога: state и update()

В loadProducts вместо render(data) теперь: allProducts = data; update();

КОД · script.js

```
const state = { category: "all", query: "" };
let allProducts = []; // заполнит loadProducts

function update() {
  const { category, query } = state; // распаковка
  let list = allProducts; // начинаем со всех
  if (category !== "all") {
    list = list.filter((p) => p.category === category);
  }
  list = list.filter((p) =>
    p.title.toLowerCase().includes(query));
  render(list);
}
```

## КАК ЭТО РАБОТАЕТ

Пользователь нажал «Аудио»

↓

state.category = "audio"

↓

update(): 6 товаров → filter → 2

↓

render(list)

↓

На странице: Наушники, Колонка

**Как это в C# / Python:** как игровой цикл: есть состояние, по нему рисуем кадр. Поменял данные — перерисовал.

ПРАКТИКА · ШАГ 2 ИЗ 3

# Кнопки категорий и живой поиск

HTML: <button data-category="audio">Аудио</button> … и <input id="search">

КОД · script.js

```
const buttons = document.querySelectorAll("[data-category]");
buttons.forEach((btn) ⇒ {
  btn.addEventListener("click", () ⇒ {
    state.category = btn.dataset.category; // "audio"
    update();
  });
});

const search = document.querySelector("#search");
search.addEventListener("input", () ⇒ {
  state.query = search.value.trim().toLowerCase();
  update();
});
```

## ЧТО ПРОИЗОЙДЁТ

**ДО**

[Все] [Аудио] [ПК] [Игры]

6 карточек — весь каталог

**КЛИК «АУДИО»**

Наушники · Колонка

**+ ВВОД «Кол »**

Колонка

**data-category="audio"** читается как **btn.dataset.category**. **trim** убирает пробелы, **toLowerCase** — регистр.

**Как это в C# / Python:** foreach (var b in buttons) b.Click += … или for b in buttons: b.config(command=…).

ПРАКТИКА · ШАГ 3 ИЗ 3

# Сортировка и «ничего не найдено»

В state добавили sort: "none", в HTML — <select id="sort"> с пунктами none и cheap.

КОД · script.js

```
// 1) В update(), прямо перед render(list):
if (state.sort === "cheap") {
  list = [...list].sort((a, b) => a.price - b.price);
}
// 2) Выбрали пункт в <select> — меняем state
sortSelect.addEventListener("change", () => {
  state.sort = sortSelect.value; // "cheap"
  update();
});
// 3) В render(list), сразу после очистки:
if (list.length === 0) {
  catalog.textContent = "Ничего не найдено 😳";
}
```

## ЧТО ПРОИЗОЙДЁТ

### ВЫБРАЛИ «СНАЧАЛА ДЕШЁВЫЕ»

Мышь 1490 → Колонка 2490 → Наушники 2990 → Геймпад 3490 → Клавиатура 3990 → Руль 5990

### ПОИСК «ТЕЛЕВИЗОР»

Ничего не найдено 😳

Пустой список — не ошибка: цикл просто ничего не добавит. Но пользователь должен понять, что поиск сработал.

**Как это в C# / Python:** list.OrderBy(p => p.Price) или sorted(products, key=lambda p: p["price"]).

МИНИ-ПРОЕКТ

# Твой каталог из JSON: поиск, фильтр, сортировка

## Задание: улучшаем проект из Части 1

1. Перенеси массив в data.json, добавь поле category и ещё 3 объекта (всего 8).
2. loadData() через fetch + async/await, запуск через Live Server.
3. state и update(): фильтр, поиск и сортировка → render(list).
4. Кнопки категорий с data-category, включая «Все».
5. Поиск по названию прямо при вводе.
6. <select> сортировки: минимум 2 варианта.
7. «Ничего не найдено» и счётчик «Найдено: X из Y».

## Чек-лист самопроверки

- Данные приходят из data.json, в консоли нет ошибок
- Перед fetch и response.json() стоит await
- «МЫШЬ», «мышь» и « мышь » находят одно и то же
- Фильтр и поиск работают одновременно
- «Без сортировки» возвращает исходный порядок
- Кнопка «Все» показывает весь каталог
- Пустой результат = сообщение, а не пустота

**Решение для сравнения:** папка lesson2-solution — «Книжная полка» с жанрами, поиском и сортировкой.

ПРОВЕРЬ СЕБЯ

# Три задачки без подсказок

**1**

```
const nums = [5, 1, 4];
console.log(nums.map((n) ⇒ n * 2));
console.log(nums.filter((n) ⇒ n > 3));
```

### Что выведет консоль?

Две строки. Подумай, какая длина у каждого массива.

**2**

```
async function load() {
  const res = await fetch("data.json");
  const data = res.json();
  console.log(data.length);
}
```

### Найди ошибку

Что окажется в консоли и почему? Как исправить одним словом?

**3**

```
// Отсортируй products от дорогих к дешёвым,
// не испортив исходный массив.
// Первым должен оказаться Руль.
```

### Напиши сам

Одна строка кода. Проверь в консоли: sorted[0].title.

**Ответы (не подглядывай!):** 1) [10, 2, 8] и [5, 4]. 2) нет await перед res.json() — в data Promise, будет undefined. 3) const sorted = [...products].sort((a, b) => b.price - a.price);

ИТОГИ ЧАСТИ 2

# Что ты теперь умеешь

- ✓ Отличать JSON от объекта: parse и stringify
- ✓ Загружать data.json через fetch + async/await
- ✓ Запускать сайт через Live Server (и знать запасной план)
- ✓ Работать со списками: filter, map, find, sort, includes
- ✓ Ловить события click, input и change
- ✓ Держать state и перерисовывать: update() → render()

ДАЛЬШЕ · ЧАСТЬ 3

## Удаление и память

Кнопка «Удалить» на каждой карточке, data-id и делегирование событий. **localStorage**: каталог помнит удалённое даже после перезагрузки, плюс кнопка «Сбросить».

И финал: собираем полный конкурсный каталог и шпаргалку по ошибкам.